

The Next Ravi Varma - Implementation Manual

Implementation Manual: The Next Ravi Varma

This is a hands-on walkthrough for getting the project running, start to finish. It assumes no prior familiarity with the codebase. Follow it top to bottom the first time.

0. Decide your path first

Training a LoRA needs a GPU. Everything else (dataset prep, the app UI shell, evaluation plumbing) works on a plain laptop. Pick one:

Path	What you need	Best for
A. Google Colab (recommended)	A free Google account	Actually training a model, with zero local setup
B. Your own GPU machine	An NVIDIA GPU with 8GB+ VRAM, CUDA installed	Repeat training runs, faster iteration
C. Local, CPU only	Any computer	Preparing your dataset before uploading it; browsing the code; generating images is possible but slow (minutes per image)

You can mix these: prepare your dataset locally (Path C), train on Colab (Path A), then bring the trained checkpoint back to your laptop to generate images.

1. Get the code onto your machine

Unzip the file you were given:

```
unzip the-next-ravi-varma.zip
cd the-next-ravi-varma
```

You should see folders named `configs/`, `data/`, `src/`, `scripts/`, `app/`, `notebooks/`, etc. If you don't, you unzipped in the wrong place – `cd` into the folder that directly contains `README.md`.

2. Install Python dependencies

Requires Python 3.10 or newer. Check with `python3 --version`.

Path B or C (local machine)

```
python3 -m venv .venv
source .venv/bin/activate          # on Windows: .venv\Scripts\activate
pip install -r requirements.txt
pip install -e .
```

This installs torch, diffusers, transformers, etc. It's a big download (a few GB) – that's normal. If you're on Path C (CPU only) and don't have a GPU, `pip install torch` still works, it just installs the CPU-only build.

Path A (Colab)

Don't install anything locally yet – you'll do this inside the Colab notebook in Step 5.

If something fails to install

- `bitsandbytes` and `xformers` are commented out for CPU-only machines in `requirements.txt` – they only work with an NVIDIA GPU + CUDA. If you're on Path C, that's expected and fine; the code auto-detects their absence and falls back to standard optimizers/attention.
- If `pip install -r requirements.txt` fails partway through, try installing torch by itself first (`pip install torch`), then re-run the full command.

Sanity check

```
python3 -c "import ravi_varma; print('OK!')"  
pytest -v
```

You should see `OK` and (if you didn't install the full ML stack) most tests passing – a few may be skipped if CLIP/torch aren't present, which is expected and not an error.

3. Get training images

You need a folder of images to train on. Two options:

Option 1: Auto-download public-domain paintings

```
python scripts/download_dataset.py --limit 40
```

This pulls Raja Ravi Varma reproductions from Wikimedia Commons (public domain – he died in 1906) along with licensing metadata. Needs internet access. 40 images is a reasonable starting size; you can lower or raise `--limit`.

Option 2: Use your own images

1. Put image files (`.jpg` , `.jpeg` , `.png` , `.webp`) directly into `data/raw/`.
2. Create `data/metadata/metadata.jsonl` by hand, one line per image:

```
{"image": "my_photo_01.jpg", "caption": null, "artist": "Raja Ravi Varma", "source": "my own scan", "year": null, "title": null}
```

(Leave `"caption": null` – you'll generate captions in Step 4.)

Only use images you actually have the right to use. See `data/README.md` for more on this.

Check the dataset is usable

```
python scripts/validate_dataset.py
```

Read the output. It will tell you: - How many images it found and how many are actually usable - Any corrupt files, duplicates, or images that are too small (below 384px on the short side) - Whether captions exist yet (they won't, until Step 4 – that's expected)

If it says “0 valid samples,” fix the errors it lists before moving on. Small warnings are fine.

4. Generate captions

Every training image needs a text caption. Two ways to do this:

```
# With a real image-captioning AI model (needs torch/transformers, works on CPU but slow)
python scripts/generate_captions.py --write-metadata

# Without any AI model -- instant, but very basic captions (lighting/color only)
python scripts/generate_captions.py --no-vlm --write-metadata
```

Either way, this writes a caption for every image into `data/metadata/metadata.jsonl` and into individual files under `data/captions/`. Open a couple of the `.json` files in `data/captions/` to see what was generated – if the automatic captions are poor, you can hand-edit `data/metadata/metadata.jsonl` directly (it’s a plain text file, one JSON object per line).

5. Resize images and build the training manifest

```
python scripts/prepare_dataset.py
```

This resizes every image to 512x512, crops it, and writes `data/processed/manifest.jsonl` – the file the trainer actually reads. If it reports fewer images than you expected, check the log output above it for “skipping ... no caption found” or “unreadable” messages.

If you’re headed to Colab (Path A): stop here, then upload the whole project folder (or at least `data/` and `configs/`) to Google Drive or directly into the Colab session – see Step 6.

6. Train the LoRA

Path A: Google Colab

1. Go to colab.research.google.com and upload `notebooks/03_lora_training.ipynb`.
2. **Runtime -> Change runtime type -> GPU (T4 is fine, free tier).**
3. Run the cells top to bottom. The notebook will:
 - Check the GPU is present (`nvidia-smi`)
 - Install dependencies
 - Let you mount Google Drive and copy your `data/` folder in (or clone your own git repo if you pushed one)
 - Validate -> caption -> prepare (safe to re-run even if you did this locally already)
 - Train – there’s a quick 50-step smoke test cell first; uncomment the full run once that works
 - Generate a couple of test images
 - Zip up the trained checkpoint and download it
4. Unzip the downloaded checkpoint into `checkpoints/lora/ravi_varma/` on your own machine.

Path B: Your own GPU machine

```
python scripts/train_lora.py --config configs/lora_sd15.yaml
```

First, try a fast smoke test to make sure everything works before committing to a full run:

```
python scripts/train_lora.py --config configs/lora_sd15.yaml --max-train-steps 50
```

If that finishes without errors in a few minutes, run the full version (drop `--max-train-steps`, or edit `configs/lora_sd15.yaml`’s `training.max_train_steps`, default 1500).

Training writes checkpoints periodically to `checkpoints/lora/ravi_varma/checkpoint-<step>/` and the final weights to `checkpoints/lora/ravi_varma/`. If it’s interrupted, re-running the same command resumes automatically from the latest checkpoint.

Watch training progress:

```
tensorboard --logdir outputs/logs
```

then open the URL it prints.

Path C: CPU only

Not recommended – LoRA training on CPU can take many hours to days even for a small dataset. Use Path A instead.

7. Generate images

Once `checkpoints/lora/ravi_varma/` contains files (not empty):

```
python scripts/generate.py --prompt "Arjuna standing before Krishna on the battlefield of Kurukshetra"
```

This prints the file paths of the saved image and its metadata. Check the printed metadata's `"lora_loaded"` field – if it says `false`, the script couldn't find your trained checkpoint (check `configs/generation.yaml`'s `model.lora_path` points to the right folder) and it generated with the plain base model instead, which won't look like Ravi Varma's style.

Useful flags:

```
python scripts/generate.py --prompt "..." --seed 42 # reproducible output
python scripts/generate.py --prompt "..." --steps 40 --guidance-scale 8.0
python scripts/generate.py --prompt "..." --pose-image ref.jpg --controlnet-type openpose
```

8. Use the web dashboard (optional, easier)

Instead of the command line, launch the browser UI:

```
python app/app.py
```

Open the local URL it prints (usually `http://127.0.0.1:7860`). You get: - **Generate tab** – type a scene, adjust sliders, click Generate - **Advanced Controls** – shows your current model config - **Evaluation tab** – score a generated image against your reference images - **About tab** – project background and limitations

The top of the page always shows whether your LoRA checkpoint is loaded.

9. Evaluate results (optional)

Put a handful of real Ravi Varma images (not used in training) into `data/reference/`, then:

```
python scripts/evaluate.py --input outputs/generated --reference data/reference
```

This writes `outputs/evaluations/report.csv` scoring every generated image on: - **Text-image alignment** – does the image match its prompt (CLIP score) - **Style similarity** – how close it is, in a rough embedding sense, to your reference images

These are comparative signals, not proof of quality or authenticity – treat them as one input among several (your own eye is the other one).

Troubleshooting

Symptom	Likely cause / fix
ModuleNotFoundError: No module named 'ravi_varma'	Run <code>pip install -e .</code> from the project root
Cannot run LoRA training: missing or broken packages [...]	You're missing torch/diffusers/etc, or on Path C without a real ML install – use Colab (Path A)
Dataset validation says “0 valid samples”	Check <code>data/raw/</code> actually has images, and they're at least 384px on the short side
Captions look wrong/empty	Edit <code>data/metadata/metadata.jsonl</code> by hand, or re-run <code>generate_captions.py</code> without <code>--no-trlm</code>
<code>prepare_dataset.py</code> prepared fewer images than expected	Look for “skipping ... no caption found” warnings just above – usually a caption is missing for that image
Generated images don't look like Ravi Varma's style at all	Check the printed metadata says <code>"lora_loaded": true</code> ; if <code>false</code> , your <code>configs/generation.yaml</code> <code>model.lora_path</code> isn't pointing at your trained checkpoint
CUDA out of memory during training	Lower <code>training.train_batch_size</code> (already 1 by default) or increase <code>training.gradient_accumulation_steps</code> in <code>configs/lora_sd15.yaml</code> ; make sure <code>training.gradient_checkpointing: true</code>
Colab session disconnects mid-training	Re-run <code>train_lora.py</code> – it auto-resumes from the last saved checkpoint under <code>checkpoints/lora/ravi_varma/checkpoint-<step>/</code>

Quick reference: the whole pipeline in one block

```

pip install -r requirements.txt && pip install -e .
python scripts/download_dataset.py --limit 40
python scripts/validate_dataset.py
python scripts/generate_captions.py --write-metadata
python scripts/prepare_dataset.py
python scripts/train_lora.py --config configs/lora_sd15.yaml # or do this step on Colab
python scripts/generate.py --prompt "your scene here"
python app/app.py

```